

The system is defined by a set of architectural constraints and functional requirements designed to convert structured user inputs into live, stateful 2D simulations.

- Computational evaluation, mathematical calculations, and schema validations must run on the Node.js backend, while reactive rendering and animations are restricted to the React client.
- The LLM cannot return free-form text or unstructured code; it must strictly output valid JSON adhering to an enforced JSON Schema to prevent frontend crashes.
- User prompts must be gathered through structured domain forms (specifying diagram type, constraints, data points, and animation triggers) rather than open-ended text to minimize hallucinations.
- When updating a diagram, the system must pass the existing state tree alongside change diffs, modifying only target parameters while preserving untouched nodes and connections.
- Node connectivity, directional flow, and mathematical dependencies must be verified on the server before dispatching the payload to the frontend.
- Built exclusively on the MERN stack (MongoDB, Express.js, React.js, Node.js) with Tailwind CSS, SVG/Canvas graph engines (such as React Flow), and Framer Motion.
- Renders category-specific parameter forms (flowcharts, tracking gauges, simulation trees) to capture user constraints.
- Hydrates the validated JSON payload into primitive 2D components, decision gates, connectors, and metric cards.
- Supports pan, zoom, drag-and-drop node adjustments, and dynamic edge routing.

- Binds parameter controls (e.g., sliders, inputs) to trigger immediate client-side re-renders and animated transitions.
- Allows diagrams and simulation states to be exported as clean JSON definitions or React-embeddable component packages.
- Maps structured form data into LLM system prompts with strict JSON schema definitions.
- Computes formulas, state workflows, algorithmic steps, and cumulative tracking metrics (e.g., calorie counters, algorithmic complexity).
- Validates incoming AI-generated JSON against expected graph contracts before responding to API requests.
- Manages user sessions, project permissions, and secure API endpoints.
- Stores user profiles, authentication metadata, and project ownership records.
- Persists node-edge definitions, coordinate positions, and visual properties.
- Maintains session snapshots, historical metric logs, and state iteration trees for undo/redo and non-destructive diffing.